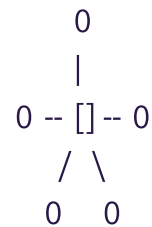


Input / Output Multiplexing:

Select function: `select()`



⇒ If one is unable to send the data then the server gets

Show more ▾

Yes. Your current notes have the **right intuition**, but one important correction will make the whole topic much clearer:

`select()` does **not** itself "unblock the server."

It lets the server **wait for multiple file descriptors simultaneously and know which ones are ready**, instead of blocking on one client.

Let's build the concept from the ground up.

1. The problem: Iterative server

Suppose your server has 3 clients:



+-----+

A simple server might do:

```
accept(client1);

recv(client1);    // wait for data
send(client1);

accept(client2);
recv(client2);
send(client2);
```

Problem

Suppose Client 1 connects but **doesn't send anything**.

The server reaches:

C

and `recv()` **blocks**.

So the server is stuck here:

```
Client 1
|
| no data
X
|
SERVER
|
X  BLOCKED
```

Meanwhile:

```
Client 2 → has data
Client 3 → has data
```

But the server cannot process them because it is stuck waiting for Client 1.

Your note should say:

Problem with blocking I/O: If the server waits for data from one client using a blocking `recv()`, it may remain blocked even though other clients are ready. Therefore, other clients cannot be serviced.

2. The solution: I/O Multiplexing

We want the server to ask:

"Which of my clients are ready right now?"

Instead of:

```
Wait for Client 1
    ↓
Wait for Client 2
    ↓
Wait for Client 3
```

we want:

```

      +-----+
Client 1 →|          |
Client 2 →|  SERVER  | → Which are ready?
Client 3 →|          |
      +-----+
```

This technique is called:

I/O Multiplexing

The main system calls used for this idea include:

- `select()`
- `poll()`
- `epoll()` — Linux
- `kqueue()` — BSD/macOS

For your syllabus, we're focusing on:

3. What does `select()` actually do?

Think of `select()` as a **watchman**.

You give it a collection of file descriptors:

```
fd 3 → Client 1
fd 4 → Client 2
fd 5 → Client 3
fd 6 → Client 4
```

and tell it:

"Watch these descriptors and tell me which ones are ready."

For example:

```

              select()
                |
      +-----+-----+
      |         |         |
    fd3      fd4      fd5
  Client1  Client2  Client3
      ↓         ↓         ↓
  ready  not ready ready

```

`select()` returns when **at least one descriptor is ready** (or when a timeout expires, or a signal interrupts it).

Then your server checks which descriptors were marked.

4. What does "ready" mean?

This is extremely important.

A descriptor can be monitored for different types of readiness.

Read readiness

"Can I read data without blocking?"

Example:

Client → sends "Hello"

Server's socket

↓

DATA AVAILABLE

↓

read-ready

This is what you'll use most often for a TCP server.

Write readiness

"Can I write/send data without blocking?"

Server → Client

↓

Can data be sent?

↓

write-ready

Exception readiness

Exceptional conditions.

This is what the third set is for:

`exceptfd`

It's much less commonly used in basic TCP server programs.

5. fd_set

Now your note:

"These are like arrays, predefined datatype is fd_set."

This needs correction.

fd_set is **not simply an array**.

It is a data type provided by the system for representing a **set of file descriptors**.

You create three sets:

```
fd_set readfds;  
fd_set writefds;  
fd_set exceptfds;
```

Think of them conceptually as:

readfds

fd 0	fd 1	fd 2	fd 3	fd 4	fd 5
↓	↓	↓	↓	↓	↓
[]	[]	[]	[✓]	[✓]	[]

Meaning:

```
fd 3 → monitor for reading  
fd 4 → monitor for reading
```

6. Important fd_set functions

You generally manipulate these sets using macros.

FD_ZERO()

Clears the set.

C

Meaning:

Start with an empty set.

`FD_SET()`

Add a descriptor to the set.

C

Meaning:

Monitor `clientfd` for read readiness.

`FD_CLR()`

Remove a descriptor.

C

`FD_ISSET()`

Check whether a descriptor was marked as ready.

```
if (FD_ISSET(clientfd, &readfds))
{
    recv(clientfd, buffer, sizeof(buffer), 0);
}
```

Meaning:

Did `select()` say that this client is ready to be read?

7. The most important concept: `select()` modifies the sets

This is something you absolutely need to remember for your exam.

Suppose initially:

```
readfds:
```

```
fd 3 → Client 1 ✓
```

```
fd 4 → Client 2 ✓
```

```
fd 5 → Client 3 ✓
```

You call:

```
C
```

Suppose only Client 2 has sent data.

After `select()`:

```
readfds:
```

```
fd 3 → Client 1 ✗
```

```
fd 4 → Client 2 ✓
```

```
fd 5 → Client 3 ✗
```

So:

```
C
```

returns true.

But:

```
C
```

returns false.

Therefore:

`select()` **modifies the `fd_set` s to contain only the descriptors that are ready.**

This is one of the most important points in the entire topic.

8. What does `select()` look like?

The basic syntax is:

```
select(  
    nfds,  
    &readfds,  
    &writefds,  
    &exceptfds,  
    &timeout  
);
```

More formally:

```
int select(  
    int nfds,  
    fd_set *readfds,  
    fd_set *writefds,  
    fd_set *exceptfds,  
    struct timeval *timeout  
);
```

9. Understanding `nfds`

This confuses almost everyone initially.

`nfds` is:

The highest-numbered file descriptor being monitored + 1

Suppose:

```
serverfd = 3
client1  = 4
client2  = 7
client3  = 9
```

Highest fd:

9

Therefore:

C

because:

$9 + 1 = 10$

Important

It is **not the number of clients**.

Wrong:

C

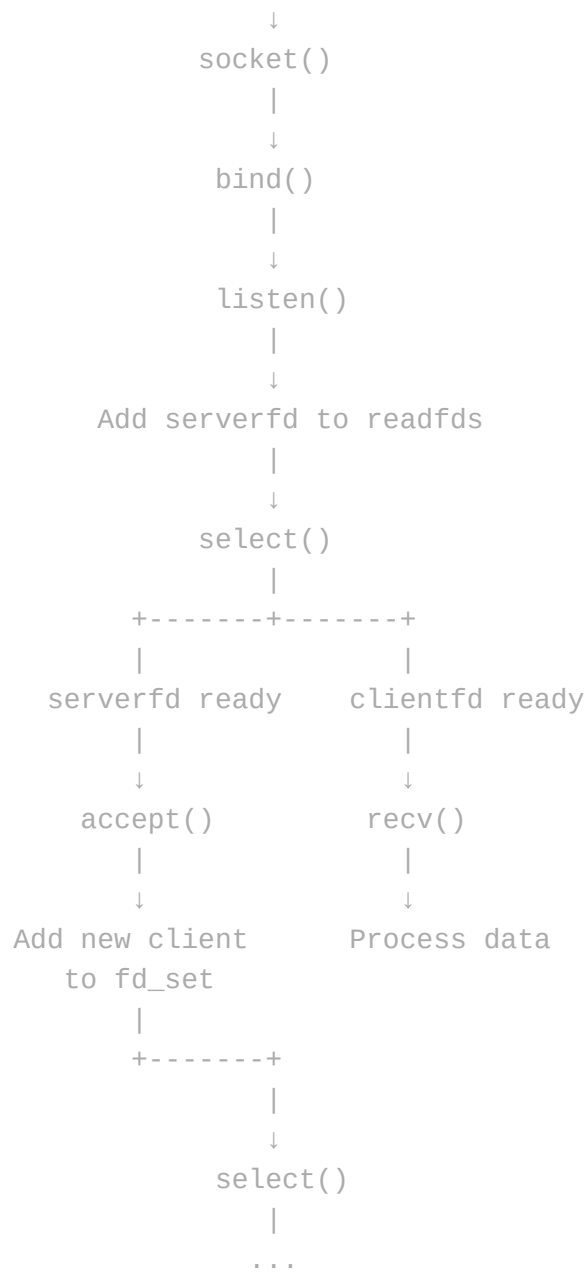
Correct:

C

10. Complete flow of a TCP server using `select()`

This is the part you should memorize conceptually.

```
SERVER SOCKET
|
```



11. Why do we put the server socket into readfds ?

This is another key idea.

A listening socket becomes **read-ready** when a new connection is waiting to be **accepted**.

So:

does **not** mean:

"Read data from the server socket."

It effectively means:

"Tell me when there is a new connection waiting."

Then:

```
if (FD_ISSET(serverfd, &readfds))
{
    clientfd = accept(serverfd, ...);
}
```

12. Client descriptors also go into readfds

Suppose:

```
serverfd = 3
```

```
client1 = 4
```

```
client2 = 5
```

```
client3 = 6
```

You put them all into the read set:

```
FD_SET(3, &readfds);
FD_SET(4, &readfds);
FD_SET(5, &readfds);
FD_SET(6, &readfds);
```

Then:

Suppose Client 2 sends data.

After `select()` :

fd	Meaning	Ready?
3	Server	No
4	Client 1	No
5	Client 2	YES
6	Client 3	No

Then:

```
if (FD_ISSET(5, &readfds))  
{  
    recv(5, ...);  
}
```

You immediately know:

Client 2 has data available.

13. What happens if multiple clients are ready?

This is where multiplexing becomes powerful.

Suppose:

```
Client 1 → data available  
Client 2 → nothing  
Client 3 → data available  
Client 4 → data available
```

`select()` returns.

The set becomes:

```
Client 1 → ✓  
Client 2 → ✗  
Client 3 → ✓
```

Client 4 → ✓

The server can process:

```
if (FD_ISSET(client1, &readfds))
    ...

if (FD_ISSET(client2, &readfds))
    ...

if (FD_ISSET(client3, &readfds))
    ...

if (FD_ISSET(client4, &readfds))
    ...
```

So one server can handle many clients without blocking on a client that isn't ready.

14. A very important correction to your original understanding

You wrote:

"At every timeinterval select() keeps a track of every client and checks which one is ready and it marks it."

Almost right, but don't think of it as continuously checking every time interval.

Better:

`select()` **waits until one or more monitored file descriptors become ready**, unless a timeout is specified.

For example:

C

means:

Wait indefinitely until something becomes ready.

With a timeout:

C

it can wait only for the specified period.

15. The actual mental model

Forget the complicated code for a moment.

Imagine a teacher supervising 50 students.

Without `select()`

Teacher:

```
"Student 1, give me your answer."
```

```
Student 1:
```

```
"I haven't finished."
```

```
Teacher waits...
```

```
Student 2:
```

```
"I have finished!"
```

```
Student 3:
```

```
"I have finished!"
```

```
Teacher doesn't care because they're stuck waiting for Student 1.
```

With `select()`

Teacher:

```
"Everyone, raise your hand when you're ready."
```

```
Student 1 → ✗
```

```
Student 2 → ✓
```

Student 3 → ✓

Student 4 → ✗

Student 5 → ✓

Teacher now handles:

Student 2

Student 3

Student 5

That's essentially the idea of I/O multiplexing.

16. Your notes — cleaned up

You can structure your notes like this:

I/O Multiplexing — `select()`

Problem

In a blocking TCP server, if the server waits for data from one client using `recv()`, it can become blocked. During this time, other clients cannot be serviced.

Solution

Use **I/O multiplexing** with `select()`.

`select()` allows the server to monitor **multiple file descriptors simultaneously** and determine which ones are ready for I/O.

`select()` **monitors three types**

`readfds` → ready for reading

`writefds` → ready for writing

`exceptfds` → exceptional conditions

The data type used is:

Important fd_set operations

```
FD_ZERO(&set);      // clear set
FD_SET(fd, &set);    // add fd
FD_CLR(fd, &set);    // remove fd
FD_ISSET(fd, &set);  // check whether fd is ready
```

Working

1. Create fd_set
- ↓
2. Add server/client FDs
- ↓
3. Call select()
- ↓
4. select() waits for readiness
- ↓
5. Ready FDs remain marked
- ↓
6. Use FD_ISSET() to identify them
- ↓
7. Perform I/O only on ready FDs

Important

select() **modifies the** fd_set s and leaves only the ready descriptors marked.

Before select():

```
fd3 ✓
fd4 ✓
fd5 ✓
fd6 ✓
```

After select():

```
fd3 ✗
fd4 ✓
fd5 ✗
fd6 ✓
```

Therefore:

C

means `fd4` is ready for reading.

`nfds`

C

It is **not the number of clients**.

17. One final distinction you should know

`select()` does **not make** `recv()` **magically non-blocking**.

Rather, it lets you do:

```
select()
  ↓
"Client 3 is ready"
  ↓
recv(Client 3)
  ↓
read data
```

Because `select()` has indicated that the descriptor is ready, the subsequent I/O operation can proceed without waiting for data that isn't there.

That distinction is important both conceptually and in viva questions.

If you understand the flow `fd_set` → `FD_SET` → `select()` → `FD_ISSET` → `recv()/accept()`, you understand the core of `select()`-based I/O multiplexing.



`</>` **select-client.c**
File

`</>` **select-server.c**
File

Explain this code line by line

You have uploaded **both the client and server programs**, so I'll explain them together. The key is to understand **what each line is doing in the `select()` architecture**, not just memorize syntax.

Part 1 — Client

The client starts by importing the libraries needed for sockets, strings, errors, etc.

select-client

1. Constants

```
#define MAXDATASIZE 100
#define STDIN 0
```

- `MAXDATASIZE` → maximum number of bytes received at once.
- `STDIN 0` → standard input has file descriptor **0**.

So:

```
STDIN = 0
```

When you type something on the keyboard, you're reading from FD `0`.

2. `main()`

```
C
```

`argc` = number of command-line arguments.

`argv` = array containing those arguments.

The program expects:

Bash

For example:

Bash

Therefore:

```
argv[0] → ./client
argv[1] → 127.0.0.1
argv[2] → 5000
```

The program checks this with:

C

because it needs **2 arguments from the user**: IP and port.

select-client

3. Create socket

C

This creates the client's socket.

`AF_INET`

IPv4.

`SOCK_STREAM`

TCP.

So:

```
AF_INET + SOCK_STREAM
      ↓
    TCP/IPv4
```

The returned value is stored in:

```
C
```

This `fd` is the client's socket file descriptor. `select-client`

Think:

```
fd = 3
```

for example.

4. Fill server address

```
C
```

We're using IPv4.

```
C
```

Suppose:

```
argv[2] = "5000"
```

First:

C

converts string → integer:

```
5000
```

Then:

C

converts the port number into **network byte order**.

C

If:

```
argv[1] = "127.0.0.1"
```

this converts the IP address into the form required by the socket API.

Then:

C

clears unused padding bytes in the structure. select-client

5. Connect

```
connect(fd, (struct sockaddr *)&server,
```

This asks:

"Connect my socket `fd` to the server described by `server`."

So:



If successful, the TCP connection is established. `select-client`

Now the important part: `select()`

The client creates:

```
fd_set rset, master, wset;
struct timeval timeout;
```

`rset` = set used by `select()` for **reading**.

`master` = permanent copy containing the FDs we want to monitor.

`wset` = intended for write monitoring, although this program doesn't actually use it.

`select-client`

6. Empty the sets

```
FD_ZERO(&rset);
FD_ZERO(&master);
```

Initially:

```
rset:
nothing
```

```
master:
nothing
```

7. Add FDs to master

```
FD_SET(fd, &master);
FD_SET(STDIN, &master);
```

This is extremely important.

Suppose:

```
fd = 3
STDIN = 0
```

Then:

```
master
|
├─ FD 0 → keyboard
└─ FD 3 → server socket
```

So the client is saying:

"I want `select()` to watch both my keyboard and my server socket."

That means the client can respond to **either**:

```
Server sends something
OR
User types something
```

8. The infinite communication loop


```
do
```

```
{
```

```
}
```

Everything inside repeats.

9. Why `rset = master`?

```
C
```

This is one of the **most important lines**.

Remember:

```
select() modifies the fd_set you give it.
```

Suppose:

```
master:
```

```
FD 0 ✓
```

```
FD 3 ✓
```

After:

```
C
```

it might become:

```
rset:
```

```
FD 0 ✓
```

```
FD 3 ✗
```

meaning:

Keyboard is ready, server isn't.

Next time around, if you don't restore it, FD 3 is gone.

Therefore:

C

restores the original list of FDs to monitor.

Memorize:

```
master = permanent list
rset   = working copy
```

10. Set timeout

```
timeout.tv_sec = 100;
timeout.tv_usec = 0;
```

~

This tells `select()` :

Wait for at most 100 seconds.

So:

```
100 seconds
  ↓
select()
  ↓
something ready? → return
nothing ready?   → timeout
```

11. The most important line

C

This is the heart of the program.

Let's break it apart.

`fd+1`

`select()` needs:

`highest FD being monitored + 1`

Here the highest FD is assumed to be the socket `fd`.

So:

C

`&rset`

Monitor these FDs for **read readiness**.

`NULL`

C

for write set → don't monitor write readiness.

Another:

C

for exception set → don't monitor exceptions.

`&timeout`

Wait for up to 100 seconds.

So conceptually:

```
select()  
|  
+---- Watch rset  
|  
+---- Don't watch write  
|  
+---- Don't watch exceptions  
|  
+---- Wait max 100 sec
```

12. Check whether server has sent data

C

This asks:

"Did `select()` mark the server socket as ready for reading?"

If yes:

C

receives data from the server. `select-client`

13. Why `buf[numbytes] = '\0' ?`

Suppose:

Server sends:

Hello

recv() gives:

H e l l o

but C strings require:

H e l l o \0

Therefore:

C

terminates the string.

Then:

C

prints it.

14. Check keyboard

Now:

C

asks:

"Did select() say that the keyboard has input available?"

If yes:

C

reads what the user typed.

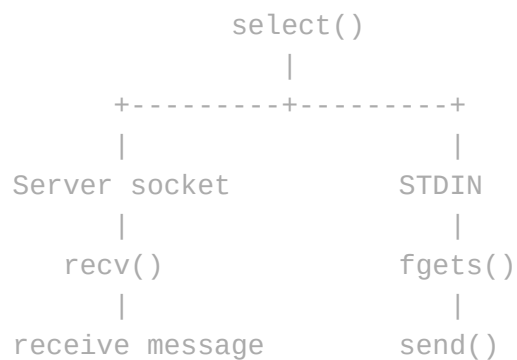
Then:

C

sends it through the socket to the server.

select-client

So the client can do both:



15. Why `select()` is useful even on the client

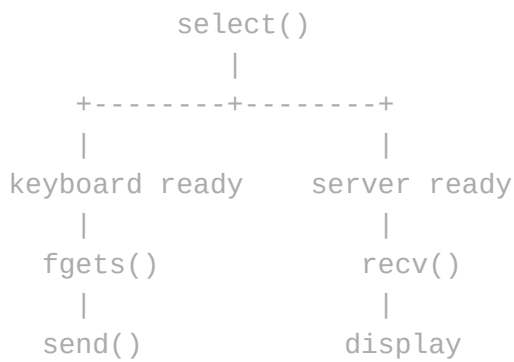
Without `select()`, you might do:

```
fgets(stdin);
send();
recv();
```

But now imagine the server sends something while you're waiting for keyboard input.

You wouldn't immediately process it.

With `select()`:



The client reacts to whichever event occurs.

16. Loop condition

C

The loop continues until the beginning of `buf` is:

```
exit
```

So if the server sends:

```
exit
```

the loop terminates.

Finally:

C

closes the socket. `select-client`

Part 2 — Server

Now the **server** is where `select()` becomes much more interesting.

The server code sets up a listening socket and then uses `select()` to monitor multiple client sockets. `select-server`

17. Server socket creation

C

Creates the server's TCP socket.

Think:

`listen_fd`

↓

Server's main/listening socket

18. Server address

C

IPv4.

C

Convert the command-line port into network byte order.

C

This means:

Accept connections arriving on the machine's local interfaces.

Then:

C

clears unused bytes.

select-server

19. bind()

```
bind(listen_fd, (struct sockaddr*)&server,  
      sizeof(struct sockaddr))
```

~

Associates:

IP + PORT

↓

listen_fd

For example:

0.0.0.0:5000

↓

listen_fd

20. listen()

C

changes the socket into a **listening socket**.

socket()

↓

bind()

↓

```
listen()
```

↓

Waiting for clients

BACKLOG = 2 means the connection queue is configured with a backlog of 2.

select-server

21. The server's FD sets

C

The important one here is:

```
all_set
```

It represents **all descriptors the server wants to monitor**.

Then:

```
FD_ZERO(&read_set);  
FD_ZERO(&write_set);  
FD_ZERO(&all_set);
```

clears them. select-server

22. client_fd[]

C

This array stores the socket FD of each client.

For example:

```
client_fd[0] = 4
```

```
client_fd[1] = 5  
client_fd[2] = 7
```

Meaning:

```
Client 1 → FD 4  
Client 2 → FD 5  
Client 3 → FD 7
```

Initially:

C

means:

This slot currently doesn't contain a client.

23. max_fd

C

This stores the highest-numbered FD currently being monitored.

Suppose:

```
listen_fd = 3  
client 1 = 4  
client 2 = 5  
client 3 = 8
```

Then:

```
max_fd = 8
```

and:

C

becomes:

C

24. Add listening socket to `all_set`

C

This says:

"Monitor the listening socket."

Why read set?

Because a listening socket becomes **read-ready when a new connection is waiting**.

25. Infinite server loop

C

The server continuously runs.

Inside:

C

Again:

```
all_set = master copy
read_set = temporary working copy
```

The server also has:

C

but notice that the actual `select()` call doesn't use `write_set`:

C

So in this version, only read readiness is actually being monitored.

26. The server's `select()`

```
ret_val = select(
    max_fd + 1,
    &read_set,
    NULL,
    NULL,
    &timeout
);
```

Translation:

"Watch every FD from 0 through `max_fd`, and tell me which ones are ready to be read. Wait at most 100 seconds."

This is the core of the entire server.

27. What can make `select()` return?

There are three relevant possibilities:

Case 1 — Error

C

The program prints:

C

and exits.

Case 2 — New client

C

is true.

Meaning:

A new connection is waiting.

So the server calls:

C

Case 3 — Existing client has data

The listening FD isn't the ready one, so the code goes into:

C

and checks the client FDs.

28. New client connection

```
conn_fd = accept(  
    listen_fd,  
    (struct sockaddr *)&client,  
    &sin_size  
);
```

`accept()` creates a **new socket specifically for that client**.

This distinction is crucial:

```
listen_fd  
  ↓  
continues listening  
  
conn_fd  
  ↓  
communication with ONE client
```

For example:

```
listen_fd = 3  
  
Client A → conn_fd = 4  
Client B → conn_fd = 5  
Client C → conn_fd = 6
```

29. Store the client FD

```
client_fd[max_index] = conn_fd;  
max_index++;  
~
```

Suppose:

```
max_index = 0
conn_fd = 4
```

Then:

```
client_fd[0] = 4
```

Next client:

```
client_fd[1] = 5
```

etc.

30. Add new client to `all_set`

C

This is **the key step**.

Before:

```
all_set:

listen_fd = 3 ✓
client 1   = 4 ✓
client 2   = 5 ✓
```

New client gets:

```
conn_fd = 6
```

After:

```
all_set:

3 ✓
4 ✓
5 ✓
```


6 ✓

Now `select()` will monitor Client 3 too.

31. Update `max_fd`

```
if(conn_fd > max_fd)
    max_fd = conn_fd;
```

Suppose:

```
max_fd = 5
conn_fd = 6
```

Then:

```
max_fd = 6
```

because next time:

C

must include FD 6.

32. Now the important client-checking loop

C

The server goes through every possible client slot.

For each one:

C

asks:

"Did `select()` say this client's socket is ready for reading?"

Suppose:

Client 1 → FD 4 → not ready

Client 2 → FD 5 → READY

Client 3 → FD 6 → not ready

Then only:

C

is true.

33. Receive client data

C

The server receives data from the ready client.

Suppose Client 2 sends:

Hello Server

Then:

`client_fd[1] = 5`

`recv(5, ...)`

↓

"Hello Server"

34. Why `numbytes > 0`?

```
C
```

means data was successfully received.

Then:

```
C
```

terminates it as a C string.

Then the server obtains the client's IP using:

```
C
```

and converts it to readable form using:

```
C
```

Then:

```
C
```

prints something like:

```
Message from client 127.0.0.1 on port 40000 :Hello Server
```

These operations are shown in the client-processing section of your server source.

```
select-server
```

35. Server sends a reply

After receiving the client's message:

```
C
fgets(buf, 1000, stdin);
```

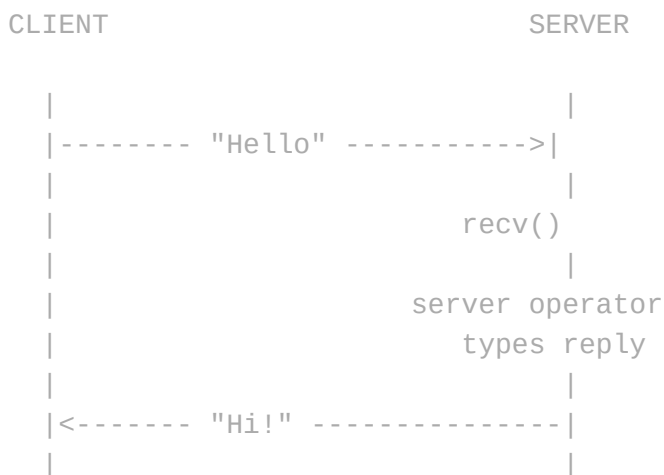
The **server operator** types a response.

Then:

```
C
send(client_fd[i], buf, strlen(buf), 0);
```

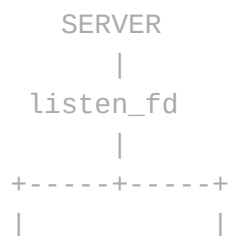
sends that response to the same client.

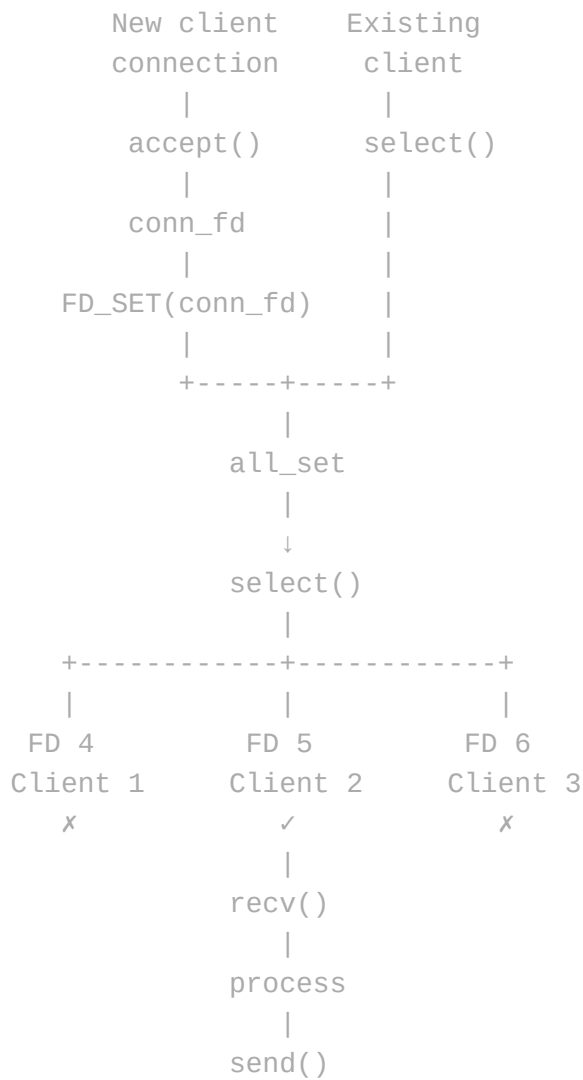
So the flow is:



36. The entire server architecture

This is the diagram I would keep in your notes:





37. The single most important thing to understand

The entire server revolves around these **three things**:

C

```
FD_SET(conn_fd, &all_set);
```

means:

Add this client to the list of clients I want to watch.

C

```
read_set = all_set;
```

means:

Make a fresh working copy because `select()` will modify it.

C

```
select(max_fd+1, &read_set, NULL, NULL, &timeout);
```

means:

Wait until one or more of those FDs are ready.

Then:

C

```
FD_ISSET(client_fd[i], &read_set)
```

means:

Was this particular client one of the FDs that became ready?

And finally:

C

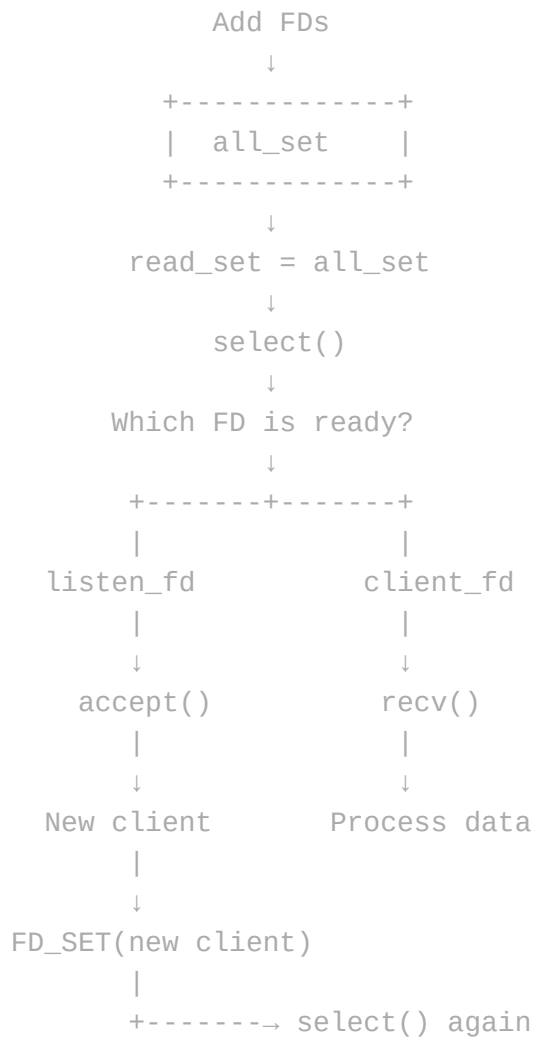
```
recv(client_fd[i], ...)
```

means:

Now safely read from that ready client.

38. The complete cycle

Memorize this:



That is **I/O multiplexing with** `select()` in your exact program.

One subtle point in your server source: `write_set` is created and copied, but the actual `select()` call passes `NULL` for the write-set parameter, so **write readiness is not actually being used in this code.**

select-server

Sources



ChatGPT can make mistakes. Check important info.